KODIGO Academy

ACTIVIDAD INDIVIDUAL · SEMANA 2

CircuitRush: Restaura la Energía

Vibecoding Guiado: Desarrollo de una Aplicación o Mini-Juego con IA como Copiloto

AI for Developers Jr. 3 — KODIGO Academy

Jaime Fernando Berrios Ortiz

Estudiante

Reporte de desarrollo, prompts y validación

https://academy.kodigo.org/

1. Descripción del proyecto

CircuitRush: Restaura la Energía es un minijuego web de lógica ambientado en una ciudad sin suministro eléctrico. El jugador debe orientar un circuito para conducir energía desde una fuente fija hasta un destino. Cada clic, o activación con Enter o Espacio, gira una pieza móvil 90 grados en sentido horario y suma un movimiento.

La propagación utiliza un recorrido en anchura (BFS): solo atraviesa casillas adyacentes cuando las conexiones coinciden en ambos extremos. Un conjunto de visitados evita recorridos repetidos y el color cian identifica las piezas energizadas. Se gana al alcanzar el destino y se pierde al agotarse el tiempo. Ambos resultados bloquean el tablero y permiten reintentar.

Existen tres niveles fijos: Encendido, de 4×4 y 120 segundos; Reconexión, de 5×5 y 180 segundos; y Sobrecarga, de 6×6 y 240 segundos. Completar uno desbloquea el siguiente. El reloj comienza con el primer giro válido y calcula el tiempo transcurrido real. Solo se puntúa al ganar:

$$P = \text{máx}(0,\ 1000 + 10s - 20m),$$

donde s son los segundos restantes y m los movimientos. Cada nivel conserva su mejor puntuación y menor número de movimientos en `localStorage`, junto con los desbloqueos; no se guarda la partida en curso.

Python y Flask sirven la página y sus archivos; HTML5 define la estructura, CSS3 presenta las piezas y JavaScript Vanilla controla la interacción. No hay cuentas, base de datos ni IA dentro del juego. La IA apoyó la programación y validación; Antigravity fue el entorno de programación asistida utilizado durante el desarrollo.

La actividad corresponde a la Semana 2 del curso AI for Developers Jr. 3 de KODIGO Academy. El trabajo presenta la evolución de una aplicación funcional, el uso progresivo de prompts y la validación crítica del código generado.

2. Evidencia de vibecoding

Se utilizaron tres prompts progresivos: funcionamiento, presentación y ampliación de jugabilidad. A continuación se explica qué se solicitó, por qué se refinó la instrucción y qué resultado se obtuvo. Los textos completos se incluyen en el material de respaldo al final del documento (página 7).

Prompt 01: desarrollo del MVP funcional

Objetivo y solicitud. Crear un tablero reproducible de 4×4, rotaciones horarias, conexiones recíprocas, propagación BFS o DFS, contador, victoria y reinicio, con Flask y JavaScript Vanilla. Exigió pruebas y resultados reales; excluyó temporizadores y ampliaciones prematuras.

Motivo y resultado. Acotar la primera versión permitió comprobar la mecánica antes de ampliarla. Se implementaron estado, conexiones, propagación y giro, además de siete pruebas iniciales. La figura 1 muestra la interfaz funcional.

Prompt 02: rediseño visual futurista y mejoras UX

Objetivo y solicitud. Inspeccionar y conservar el MVP, mejorar la jerarquía visual, los estados eléctricos, las animaciones, el modal, el teclado y la adaptación responsive, sin agregar niveles ni reloj.

Motivo y resultado. El refinamiento abordó la claridad visual sobre una lógica ya verificable. Se incorporaron la consola energética, indicadores de conexión, brillo, transiciones y foco visible, respetando movimiento reducido. La figura 2 permite comparar el resultado con el MVP.

Prompt 03: niveles, temporizador, puntuación y récords

Objetivo y solicitud. Añadir tres tamaños de tablero, soluciones verificadas mediante giros legales, reloj basado en tiempo real, puntuación explicable, récords locales, desbloqueo y resultados de victoria o derrota. Exigió conservar las siete pruebas y comprobar datos corruptos y bloqueos.

Motivo y resultado. Sobre la interfaz anterior se amplió la progresión y la repetición de partidas. El código contiene tres configuraciones resolubles, una sesión que gestiona el reloj y almacenamiento tolerante a fallos. Las figuras 3 y 4 documentan esos cambios.

3. Iteración y mejora

El desarrollo se organizó en tres etapas para controlar el alcance y conservar el funcionamiento anterior. La tabla 1 resume los cambios y las evidencias de cada una.

Tabla 1: Evolución de CircuitRush

Iteración	Mejora principal	Evidencia
01	Tablero 4 × 4, BFS, giros, victoria y reinicio.	Siete pruebas iniciales y captura del MVP.
02	Rediseño, estados eléctricos, animaciones, teclado y responsive.	Capturas Firefox y pruebas de foco y movimiento reducido.
03	Niveles, reloj, puntos, récords y desbloqueo.	Diecinueve pruebas y capturas de niveles y puntuación.

Nota. Síntesis del código y del registro de iteraciones del repositorio.

Las figuras 1 y 2 muestran el paso de los indicadores básicos a una consola energética. Se conservaron fuente, destino y rotación.



Figura 1: MVP: tablero inicial de 16 piezas

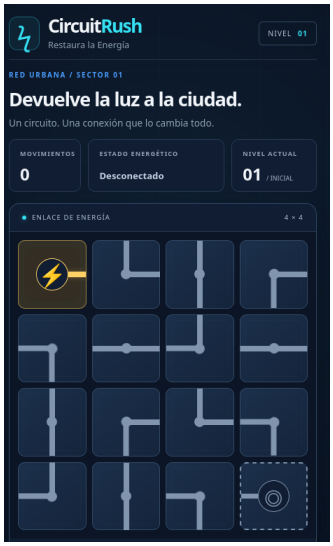


Figura 2: Consola rediseñada en la iteración 02

Nota. Capturas propias del repositorio. Se recortan márgenes exteriores para facilitar la comparación y se conserva la proporción.

La tercera iteración añade selector, bloqueos y reloj. La figura 3 muestra Reconexión; Sobrecarga amplía el tablero a 6×6. Las pruebas verifican que los tres niveles empiezan sin resolver y que una secuencia legal alcanza el destino. No se afirma que esas soluciones sean mínimas.

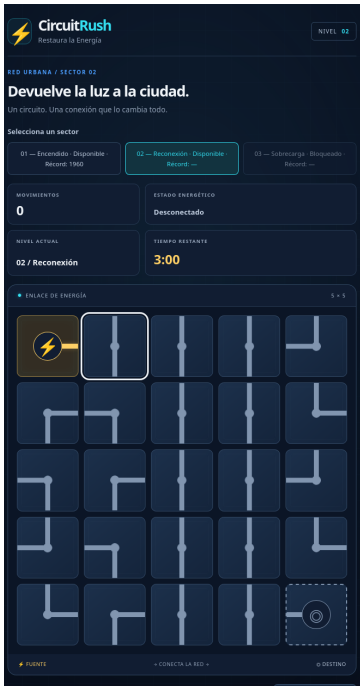


Figura 3: Reconexión: tablero 5×5, selector y reloj



Figura 4: Victoria con 12 movimientos y 1960 puntos

Nota. Capturas 10 y 13 de Firefox, recortadas sin deformación. Se conserva el tablero completo y el modal de resultado. La puntuación mostrada es $1000 + 120(10) - 12(20) = 1960$.

4. Validación del resultado

Resultados actuales. Se ejecutó nuevamente el comando solicitado:

```
node --test tests/game.test.cjs
```

Terminó correctamente y resumió **1 archivo aprobado, 0 fallidos y 0 omitidos** en este entorno. La ejecución complementaria `node tests/game.test.cjs` detalló **19 pruebas aprobadas, 0 fallidas y 0 omitidas**. Se distingue el conteo por archivo del de casos. Los registros de ejecución respaldan estos resultados.

La suite verifica giros, movimientos, conexiones recíprocas, ciclos, victoria, bloqueo y reinicio; valida los tres tamaños y sus soluciones mediante giros legales. Con reloj controlado comprueba inicio, decremento, parada, reinicio e intervalo único. También cubre puntuación no negativa, récords, recarga simulada, datos corruptos, niveles bloqueados y desbloqueo. El DOM simulado valida controles; no sustituye un navegador.

Ejecución local. `.venv/bin/python app.py` sirve el juego en `http://127.0.0.1:5000`. En esta revisión, el cliente Flask obtuvo HTTP 200 para página, CSS y JavaScript. No se presenta como una interacción visual nueva.

Evidencia histórica y ajuste. Las iteraciones 02 y 03 registran automatización en Firefox de teclado, victoria, continuación, recarga y movimiento reducido. En la segunda, Tab permitía abandonar el botón del modal: se añadió manejo de teclado y la repetición documentada aprobó. La tercera amplió ese manejo a la continuación. Otro ajuste fue usar una esquina en fila 4, columna 3 para permitir el recorrido; la suite actual verifica su solución de 12 movimientos.

Alcance y límites. Las mediciones históricas no detectaron desbordamiento a 1366, 768 y 500 píxeles. Las capturas de 390 usaron una configuración de prueba; la derrota en Firefox utilizó reloj inyectado. No se repitió esa automatización al redactar este reporte. No existen evidencias de interacción táctil física, lector de pantalla, otros navegadores o suspensión real de pestaña por el sistema operativo. Las pruebas temporales simulan retrasos. No se confunden estas comprobaciones automatizadas con pruebas manuales, ni se afirma accesibilidad total.

5. Reflexión final

Propuesta para revisión personal. Las respuestas siguientes requieren la revisión del autor para confirmar que describen su aprendizaje y su comprensión del código.

a. ¿Qué aprendí utilizando IA para programar?

Aprendí a dividir una solicitud en cambios comprobables: primero el circuito, después la interfaz y finalmente los niveles y el reloj. Pedir una función no fue suficiente; también necesité establecer cómo comprobarla, por ejemplo mediante conexiones recíprocas y una secuencia de giros que alcanzara el destino.

b. ¿Qué ventajas y limitaciones identifiqué del vibecoding?

La ventaja que identifico es convertir instrucciones concretas en una implementación y revisarla por etapas. La limitación es que una respuesta o captura no demuestra todo el funcionamiento. Las pruebas con un reloj simulado ayudan a revisar el vencimiento, pero no sustituyen la interacción en un móvil físico ni la revisión del código generado.

c. ¿Qué partes del código comprendo y cuáles necesito reforzar?

Puedo explicar la separación entre Flask, la interfaz y el estado del juego, así como la relación entre un giro y el contador. Necesito reforzar BFS, el control del tiempo y la validación del almacenamiento. Antes de presentar esta respuesta como definitiva, debo confirmar cuáles de estas partes comprendo realmente y demostrarlas con el código.

Material de respaldo: prompts utilizados

Los siguientes textos reproducen íntegramente los tres prompts de desarrollo, en orden. Se conservan sus instrucciones y restricciones originales; las limitaciones de cada etapa deben interpretarse dentro de esa iteración. La tipografía compacta permite mantener separado este respaldo del reporte principal.

Prompt 01: texto completo

```
# PROMPT 01 — CIRCUITRUSH: DESARROLLO DEL MVP FUNCIONAL

## 1. Rol y contexto

Actúa como desarrollador full-stack senior, especialista en
Python, Flask, HTML5, CSS3, JavaScript y desarrollo de videojuegos
web 2D, con experiencia en programación asistida por inteligencia
artificial y buenas prácticas de ingeniería de software.

Desarrollaremos **CircuitRush: Restaura la Energía**, un minijuego
web de lógica y resolución de circuitos eléctricos, como parte de
la actividad individual **«Vibecoding Guiado: Desarrollo de una
Aplicación o Mini-Juego con IA como Copiloto»**, correspondiente
al bootcamp AI for Developers Jr. 3 de KODIGO Academy.

El objetivo académico es demostrar el uso de IA como asistente de
programación mediante prompts progresivos, iteraciones
justificadas, comprensión del código, validación crítica y
documentación del proceso.

La IA debe actuar como copiloto técnico, no como sustituto del
criterio del desarrollador.

**Importante:** esta es la primera iteración de varias. No debes
implementar todas las funcionalidades futuras ni generar un
proyecto excesivamente complejo.

## 2. Concepto del videojuego

CircuitRush es un minijuego de lógica ambientado en una ciudad
futurista que ha sufrido un apagón eléctrico.

El jugador debe restablecer el suministro de energía mediante la
rotación de piezas de circuitos distribuidas sobre un tablero.

Cada pieza contiene conexiones eléctricas que pueden orientarse en
distintas direcciones. Al hacer clic sobre una pieza, esta gira 90
grados en sentido horario.

La energía se transmite desde una fuente eléctrica hacia las
piezas correctamente conectadas hasta alcanzar el nodo de destino.
```

Objetivo de la primera versión: conseguir que el jugador pueda completar un circuito funcional desde la fuente hasta el destino mediante una interfaz web interactiva.

3. Stack tecnológico obligatorio

Utiliza las siguientes tecnologías:

- Python 3 para el backend.
- Flask como framework web.
- HTML5 para la estructura.
- CSS3 para la presentación visual.
- JavaScript Vanilla para la lógica interactiva del videojuego.
- Git para el control de versiones.

Restricciones:

- No utilizar React, Vue, Angular ni frameworks frontend adicionales.
- No implementar bases de datos en esta iteración.
- No incorporar autenticación ni cuentas de usuario.
- No utilizar servicios externos ni APIs de terceros.
- No integrar modelos de IA dentro del videojuego.
- No incorporar Docker, microservicios ni arquitecturas complejas.
- Evitar dependencias innecesarias.

La aplicación debe ejecutarse localmente y funcionar sin conexión a Internet una vez instaladas sus dependencias.

4. Arquitectura del proyecto

Utiliza una estructura sencilla, organizada y comprensible:

```text
circuitrush/
├── app.py
├── requirements.txt
├── README.md
├── templates/
│ └── index.html
├── static/
│ ├── css/
│ │ └── style.css
│ └── js/
│ └── game.js
└── docs/
└── iteraciones.md
```

Responsabilidades:

app.py: inicialización de Flask y ruta principal para servir el videojuego.

index.html: estructura de la interfaz.

style.css: estilos, diseño visual y adaptación a distintos tamaños de pantalla.

game.js: representación del tablero, rotación de piezas, conexiones eléctricas, propagación de energía y condiciones de victoria.

iteraciones.md: registro de los cambios, decisiones técnicas, pruebas y evidencias del desarrollo.

Mantén una separación clara de responsabilidades.

5. Funcionalidades obligatorias del MVP

5.1 Pantalla principal

Implementar una interfaz que contenga:

- Nombre del videojuego: CircuitRush.
- Subtítulo: Restaura la Energía.
- Breve instrucción para el jugador.
- Tablero interactivo.
- Indicador de movimientos.
- Estado de conexión eléctrica.
- Botón para reiniciar la partida.

5.2 Tablero

Crear un tablero de 4 × 4 casillas.

Cada casilla debe representar una pieza de circuito eléctrico.

Implementar inicialmente:

- Piezas rectas.
- Piezas en forma de esquina.
- Piezas en forma de T, si resultan necesarias.
- Una fuente de energía.
- Un nodo de destino.

Representar cada pieza mediante CSS o SVG local, evitando imágenes externas.

5.3 Rotación

Cuando el usuario haga clic sobre una pieza giratoria:

1. Girarla 90 grados en sentido horario.
2. Actualizar su orientación interna.
3. Incrementar el contador de movimientos.
4. Recalcular las conexiones eléctricas.
5. Actualizar visualmente el tablero.

La fuente y el destino deben permanecer fijos en esta primera versión.

5.4 Propagación eléctrica

Implementar un algoritmo sencillo de recorrido de grafos, como BFS o DFS.

Cada pieza debe definir las direcciones hacia las que posee conexiones: arriba, derecha, abajo e izquierda.

La energía únicamente puede pasar entre dos casillas adyacentes cuando ambas tienen conexiones orientadas una hacia la otra.

El algoritmo debe:

- Comenzar desde la fuente.
- Comprobar conexiones bidireccionales válidas.
- Evitar recorrer indefinidamente los mismos nodos.
- Identificar todas las piezas que reciben energía.
- Actualizar su estado visual.
- Determinar si la energía alcanza el destino.

Una pieza no debe considerarse energizada simplemente por estar junto a otra.

5.5 Condición de victoria

Cuando la energía alcance el destino:

- Mostrar un mensaje de victoria.
- Mostrar el número de movimientos realizados.
- Impedir movimientos adicionales hasta reiniciar.
- Permitir comenzar nuevamente.

5.6 Reinicio

El botón de reinicio debe:

- Restaurar las orientaciones iniciales.
- Reiniciar el contador de movimientos.
- Eliminar el estado de victoria.
- Recalcular la propagación eléctrica.
- Permitir jugar nuevamente.

5.7 Nivel inicial

Implementar un único nivel prediseñado.

Requisitos:

- Debe tener al menos una solución válida.
- Debe comenzar sin una conexión completa entre fuente y destino.
- Debe poder resolverse mediante rotaciones.
- Debe ser reproducible para facilitar las pruebas.

No generar niveles aleatorios todavía.

```
## 6. Diseño visual inicial

Crear una interfaz moderna inspirada en sistemas eléctricos
futuristas.

Paleta sugerida:

- Azul oscuro para el fondo.
- Azul cian para los circuitos energizados.
- Gris azulado para las conexiones inactivas.
- Amarillo para la fuente de energía.
- Verde para indicar victoria.

Requisitos visuales:

- Diseño limpio y profesional.
- Tablero centrado.
- Piezas claramente diferenciadas.
- Estados visuales identificables.
- Transiciones suaves de rotación.
- Interfaz adaptable a escritorio y dispositivos móviles.
- Tipografía legible.
- Buen contraste entre texto y fondo.

Prioriza la claridad y la funcionalidad. Los efectos visuales
avanzados se desarrollarán en una iteración posterior.

## 7. Requisitos de calidad

Aplicar las siguientes prácticas:

- Código modular y legible.
- Funciones con responsabilidades definidas.
- Nombres descriptivos para variables y funciones.
- Validaciones para evitar estados incorrectos.
- Manejo básico de errores.
- Sin código duplicado innecesario.
- Sin dependencias que no aporten valor.
- Comentarios únicamente cuando expliquen lógica no evidente.

No añadir funcionalidades que no hayan sido solicitadas.

## 8. Validación obligatoria

Antes de declarar terminada la implementación, comprobar los
siguientes escenarios:

**Prueba 01:** Flask inicia correctamente y muestra la aplicación.

**Prueba 02:** el tablero contiene exactamente 16 casillas.

**Prueba 03:** hacer clic sobre una pieza giratoria modifica su
orientación 90 grados.

**Prueba 04:** cada rotación válida incrementa el contador una
sola vez.
```

Prueba 05: dos piezas con conexiones incompatibles no transmiten energía.

Prueba 06: dos piezas adyacentes con conexiones compatibles transmiten energía cuando existe un camino energizado desde la fuente.

Prueba 07: el algoritmo no entra en un ciclo infinito ante circuitos cerrados.

Prueba 08: existe una secuencia de movimientos que permite completar el nivel.

Prueba 09: al alcanzar el destino aparece el mensaje de victoria.

Prueba 10: reiniciar restaura correctamente el estado inicial.

Prueba 11: la interfaz sigue siendo utilizable en pantallas pequeñas.

Implementa pruebas automatizadas de la lógica pura cuando resulte sencillo. Complementa con pruebas manuales de la interfaz.

No declares que una prueba pasó si no fue ejecutada. Distingue entre pruebas ejecutadas, pendientes y no verificables desde el entorno.

9. Documentación académica

Esta actividad será evaluada según cuatro criterios:

1. Funcionamiento de la aplicación: 2.5 puntos.
2. Uso estratégico de IA y vibecoding: 2.5 puntos.
3. Iteración y mejora de prompts: 3 puntos.
4. Comprensión y validación del código: 2 puntos.

En `docs/iteraciones.md`, documenta la primera iteración incluyendo:

- Objetivo de la iteración.
- Prompt utilizado.
- Funcionalidades implementadas.
- Decisiones técnicas.
- Archivos creados.
- Pruebas ejecutadas y resultados reales.
- Errores encontrados, si los hubo.
- Correcciones aplicadas.
- Limitaciones conocidas.
- Funcionalidades pendientes.

No inventes errores, resultados, capturas ni evidencias.

La reflexión personal del estudiante se elaborará posteriormente y no debe redactarse como si ya hubiese sido realizada.

```
## 10. Metodología de trabajo

Trabaja en este orden:

**Fase 1 — Análisis**

Inspecciona el directorio de trabajo y determina si ya existe un proyecto.

Explica brevemente la arquitectura propuesta y las responsabilidades de los archivos.

**Fase 2 — Implementación**

Construye únicamente el MVP descrito.

**Fase 3 — Validación**

Ejecuta las pruebas posibles y comunica sus resultados.

**Fase 4 — Documentación**

Actualiza el README y el registro de iteraciones con información verificable.

**Fase 5 — Entrega de resultados**

Presenta un resumen conciso con:

- Archivos creados o modificados.
- Funcionalidades completadas.
- Pruebas realizadas.
- Errores y correcciones.
- Limitaciones pendientes.
- Comandos necesarios para ejecutar el proyecto.

## 11. Instrucciones finales

No avances hacia la segunda iteración sin mi autorización.

No agregues temporizadores, puntuaciones avanzadas, niveles adicionales, efectos de sonido, autenticación ni funcionalidades ajenas al MVP.

No modifiques archivos externos al proyecto.

No realices commits ni pushes automáticamente.

Prioriza que el videojuego funcione correctamente y que yo pueda comprender, explicar y validar su código.

**Comienza con el análisis de la estructura y continúa con la implementación de la primera versión funcional de CircuitRush.**
```

Prompt 02: texto completo

```
# PROMPT 02 — CIRCUITRUSH: REDISEÑO VISUAL FUTURISTA Y MEJORAS UX

## 1. Contexto del proyecto

Actúa como desarrollador full-stack senior y diseñador UI/UX
especializado en videojuegos web 2D, utilizando Python, Flask,
HTML5, CSS3 y JavaScript Vanilla.

Estamos desarrollando **CircuitRush: Restaura la Energía**, un
minijuego académico para la actividad «Vibecoding Guiado:
Desarrollo de una Aplicación o Mini-Juego con IA como Copiloto»,
del bootcamp AI for Developers Jr. 3 de KODIGO Academy.

La Iteración 01 ya fue implementada, probada manualmente y
publicada en GitHub.

Actualmente, el videojuego permite:

- Visualizar un tablero de circuitos eléctricos.
- Girar piezas 90 grados mediante clics.
- Propagar energía a través de conexiones válidas.
- Contabilizar movimientos.
- Detectar la victoria.
- Reiniciar la partida.

El Prompt 01 se encuentra documentado en `info.md`.

**Objetivo de esta iteración:** transformar la interfaz inicial en
una experiencia visual futurista, moderna, atractiva y responsive,
conservando el funcionamiento existente.

## 2. Instrucción inicial obligatoria

Antes de modificar cualquier archivo:

1. Inspecciona la estructura actual del proyecto.
2. Lee `info.md`, `README.md` y `docs/iteraciones.md`, si existen.
3. Analiza `app.py`, `templates/index.html`,
`static/css/style.css` y `static/js/game.js`.
4. Identifica las funciones responsables de rotación, propagación
eléctrica, victoria y reinicio.
5. Determina qué archivos requieren modificaciones.
6. Conserva el comportamiento funcional de la Iteración 01.

No reconstruyas el proyecto desde cero.

No reemplaces innecesariamente la lógica de juego que ya funciona.

## 3. Dirección artística

Diseña una interfaz inspirada en paneles de control energéticos,
videojuegos de ciencia ficción y sistemas eléctricos futuristas.

Utiliza una identidad visual coherente.
```

Paleta de colores

- Fondo principal: `#080F1E`.
- Superficies: `#111D32`.
- Paneles secundarios: `#192B44`.
- Cian eléctrico: `#35E0F2`.
- Azul luminoso: `#4D9FFF`.
- Amarillo energético: `#FFD166`.
- Verde de éxito: `#4ADE80`.
- Texto principal: `#F1F5F9`.
- Texto secundario: `#94A3B8`.

Puedes ajustar ligeramente estos colores cuando sea necesario para mejorar contraste y accesibilidad.

Evita colores excesivamente saturados y efectos visuales que dificulten la lectura.

4. Rediseño de la interfaz

4.1 Encabezado

Crear un encabezado compacto con:

- Identidad visual de CircuitRush.
- Nombre del videojuego.
- Subtítulo «Restaura la Energía».
- Indicador de nivel actual: Nivel 01.

Utilizar una tipografía moderna y legible.

No depender de fuentes externas que requieran conexión a Internet.

4.2 Panel de estadísticas

Diseñar tarjetas o indicadores para:

- Movimientos realizados.
- Estado energético: desconectado, parcialmente conectado o restaurado.
- Nivel actual.

Los indicadores deben actualizarse según el estado real del juego.

No implementar todavía puntuaciones avanzadas ni temporizadores.

4.3 Tablero eléctrico

El tablero debe ser el elemento visual principal.

Requisitos:

- Diseño centrado.
- Casillas uniformes.
- Bordes sutiles.
- Separación consistente entre piezas.
- Circuitos claramente visibles.

- Estados visuales diferenciados.
- Indicadores reconocibles de fuente y destino.
- Respuesta visual al pasar el cursor sobre una pieza giratoria.
- Diseño adaptable a diferentes resoluciones.

Las piezas deben conservar su orientación y comportamiento actuales.

4.4 Estados eléctricos

Representar visualmente tres estados:

Sin energía: conexiones en gris azulado.

Con energía: conexiones iluminadas en cian eléctrico.

Fuente: identificador amarillo.

Cuando una conexión reciba energía, mostrar un brillo moderado.

La iluminación debe corresponder exactamente al estado calculado por el algoritmo existente.

4.5 Animación de rotación

Agregar una transición visual suave al girar una pieza.

Duración sugerida: entre 150 y 250 milisegundos.

Requisitos:

- La animación no debe alterar la lógica.
- El estado visual debe coincidir con la orientación interna.
- Evitar saltos visuales.
- No bloquear permanentemente la interacción.
- Prevenir inconsistencias ante clics rápidos.

4.6 Efectos de propagación eléctrica

Incorporar un efecto visual discreto cuando una pieza pase de inactiva a energizada.

Puede utilizarse:

- Resplandor.
- Cambio progresivo de color.
- Transición de intensidad.

No utilizar animaciones excesivas ni bibliotecas adicionales.

Respetar la preferencia `prefers-reduced-motion`.

4.7 Pantalla de victoria

Sustituir o mejorar el mensaje actual mediante un modal visual integrado.

Debe mostrar:

- Icono de energía restaurada.
- Mensaje «¡Energía restaurada!».

- Número de movimientos realizados.
- Botón «Jugar nuevamente».

El modal debe:

- Aparecer únicamente al completar el nivel.
- Permitir reiniciar correctamente.
- Ser accesible mediante teclado.
- Mantener un contraste adecuado.
- No impedir que el usuario comprenda el resultado.

4.8 Controles

Rediseñar el botón de reinicio.

Incluir estados visuales para:

- Normal.
- Hover.
- Focus.
- Presionado.

No agregar controles innecesarios.

4.9 Instrucciones

Incluir una sección breve con las instrucciones:

«Haz clic en las piezas para girarlas 90°. Conecta la fuente de energía con el destino para restaurar el suministro eléctrico».

Agregar una pequeña leyenda visual para identificar los estados de energía.

5. Diseño responsive

La interfaz debe funcionar correctamente en:

- Escritorio: 1920 × 1080.
- Laptop: 1366 × 768.
- Tablet: 768 × 1024.
- Móvil: 390 × 844.

Requisitos:

- Sin desplazamiento horizontal.
- Tablero completo visible en el ancho disponible.
- Piezas suficientemente grandes para interacción táctil.
- Paneles reorganizados en pantallas pequeñas.
- Texto legible.
- Espaciado consistente.
- Modal adaptable.

En pantallas pequeñas se permite desplazamiento vertical cuando sea necesario.

Prioriza que el tablero sea cómodo de utilizar.

```
## 6. Accesibilidad y experiencia de usuario

Implementar mejoras básicas:

- Botones semánticos para las piezas interactivas, cuando sea
apropiado.
- Navegación mediante teclado.
- Indicadores visibles de foco.
- Etiquetas accesibles para fuente, destino y piezas.
- Mensajes de estado comprensibles.
- Contraste suficiente.
- Compatibilidad con `prefers-reduced-motion`.

No depender exclusivamente del color para comunicar estados
importantes.

## 7. Restricciones técnicas

Mantener:

- Python.
- Flask.
- HTML5.
- CSS3.
- JavaScript Vanilla.

No incorporar:

- React, Vue ni Angular.
- Tailwind o Bootstrap.
- Bases de datos.
- Autenticación.
- APIs externas.
- Nuevos niveles.
- Temporizadores.
- Sistemas avanzados de puntuación.
- Nuevas mecánicas de juego.

Priorizar CSS y SVG existentes para la presentación visual.

Mantener el proyecto sencillo, organizado y comprensible.

## 8. Validación funcional

Después del rediseño, comprobar:

1. Flask inicia correctamente.
2. El tablero mantiene sus 16 casillas.
3. Las piezas giran 90 grados.
4. Cada rotación válida incrementa el contador exactamente una
vez.
5. La propagación eléctrica sigue funcionando.
6. Las conexiones incompatibles no transmiten energía.
```

7. El nivel sigue siendo resoluble.
8. La victoria se detecta correctamente.
9. El modal muestra los movimientos reales.
10. Reiniciar restaura el estado inicial.
11. Los clics rápidos no producen inconsistencias.
12. La interfaz es utilizable en las resoluciones solicitadas.
13. La navegación por teclado funciona.
14. Las animaciones reducidas se respetan.

Ejecuta las pruebas automatizadas disponibles y realiza las comprobaciones visuales que permita el entorno.

No declares superadas pruebas que no hayas ejecutado.

Si no puedes verificar visualmente una resolución, indícalo como pendiente de validación manual.

9. Evidencias para KODIGO

Esta iteración debe demostrar una mejora clara respecto a la primera versión.

Conserva las evidencias de la Iteración 01.

Organiza las nuevas capturas en una carpeta correspondiente a la Iteración 02, siguiendo la estructura documental existente.

Capturas sugeridas:

- Captura 05: interfaz rediseñada.
- Captura 06: circuito parcialmente energizado.
- Captura 07: modal de victoria.
- Captura 08: visualización responsive.

Si puedes generar capturas reales mediante las herramientas disponibles, hazlo.

Si no puedes, indica exactamente cuándo y dónde debo tomarlas manualmente.

No inventes evidencias.

10. Documentación de la iteración

Actualiza `docs/iteraciones.md` incorporando:

Iteración 02 — Rediseño visual futurista y mejoras UX

Documentar:

1. Objetivo de la iteración.
2. Referencia al Prompt 02.
3. Justificación de la mejora.
4. Estado visual anterior.
5. Cambios implementados.
6. Archivos modificados.
7. Pruebas ejecutadas.

```
8. Errores identificados y correcciones reales.
9. Comparación antes y después.
10. Limitaciones pendientes.

Actualiza el README únicamente cuando sea necesario.

Respecto a `info.md`:

- Conserva íntegramente el Prompt 01.
- No elimines ni sobrescribas información anterior.
- Incorpora el Prompt 02 en una sección claramente identificada.
- Mantén ambos prompts en orden cronológico.
- Si `info.md` tiene una estructura particular, respétala.

La documentación debe reflejar acciones realmente realizadas.

No redactes una reflexión personal ficticia en nombre del estudiante.

## 11. Forma de trabajo

Trabaja en cuatro fases:

**Fase A — Análisis:** inspeccionar el proyecto y detectar las áreas visuales que necesitan mejoras.

**Fase B — Implementación:** aplicar el rediseño conservando las mecánicas.

**Fase C — Validación:** ejecutar pruebas y corregir regresiones detectadas.

**Fase D — Documentación:** registrar cambios y evidencias verificables.

Al finalizar, entrega un resumen técnico breve con:

- Archivos modificados.
- Mejoras visuales implementadas.
- Pruebas ejecutadas y sus resultados.
- Errores corregidos.
- Evidencias disponibles.
- Aspectos pendientes de validación manual.

## 12. Restricciones finales

No reconstruyas CircuitRush desde cero.

No alteres el algoritmo eléctrico salvo que identifiques un defecto real y justifiques su corrección.

No realices commits ni pushes automáticamente.

No avances a la Iteración 03.

No introduzcas sobreingeniería.

**Comienza inspeccionando el proyecto existente y ejecuta únicamente la Iteración 02.**
```

Prompt 03: texto completo

```
# PROMPT 03 — CIRCUITRUSH: NIVELES PROGRESIVOS, TEMPORIZADOR Y
SISTEMA DE PUNTUACIÓN

## 1. Rol y contexto

Actúa como desarrollador full-stack senior especializado en
Python, Flask, JavaScript Vanilla, HTML5, CSS3 y videojuegos web
interactivos.

Continuaremos el desarrollo de **CircuitRush: Restaura la
Energía**, proyecto académico del bootcamp AI for Developers Jr. 3
de KODIGO Academy.

El proyecto utiliza Python, Flask, HTML, CSS y JavaScript, y
cuenta con dos iteraciones completadas:

- **Iteración 01:** tablero funcional, rotación de piezas,
propagación eléctrica, victoria y reinicio.
- **Iteración 02:** rediseño visual futurista, efectos de
iluminación, animaciones, interfaz responsive y mejoras de
accesibilidad.

El código se encuentra versionado mediante Git y GitHub.

El objetivo de esta tercera iteración es ampliar la jugabilidad
incorporando niveles progresivos, temporizador, puntuación y
récords locales.

**Restricción principal:** conservar la identidad visual y las
funcionalidades existentes. No reconstruir el proyecto.

## 2. Inspección inicial obligatoria

Antes de modificar el código:

1. Examina la estructura actual.
2. Lee `README.md`, `docs/datos/info.md` y `docs/iteraciones.md`.
3. Inspecciona los archivos HTML, CSS y JavaScript.
4. Identifica las funciones actuales de rotación, propagación
eléctrica, victoria y reinicio.
5. Examina las pruebas automatizadas existentes en
`tests/game.test.cjs`.
6. Ejecuta las pruebas actuales para establecer una línea base.

No elimines ni reemplaces pruebas existentes sin una justificación
técnica.

## 3. Sistema de niveles

Implementar tres niveles:

| Nivel | Tablero | Tiempo inicial |
|---|---|---|
| 01 — Encendido | 4×4 | 120 segundos |
```

```
| 02 — Reconexión | 5×5 | 180 segundos |
| 03 — Sobrecarga | 6×6 | 240 segundos |
```

Cada nivel debe tener una configuración reproducible y una solución válida.

Requisitos

- Mantener el nivel 01 existente siempre que sea posible.
- Crear dos nuevos tableros.
- Adaptar dinámicamente la cuadrícula.
- Incrementar progresivamente la dificultad.
- Evitar configuraciones imposibles.
- Mantener las piezas dentro de los límites del tablero.
- Bloquear inicialmente los niveles posteriores.
- Desbloquear el siguiente nivel al completar el anterior.
- Permitir repetir niveles desbloqueados.

La fuente y el destino deben identificarse claramente.

Importante: implementar una comprobación automatizada que confirme que cada tablero es resoluble mediante rotaciones válidas.

No generar tableros aleatorios en esta iteración.

4. Temporizador

Implementar una cuenta regresiva visible.

Comportamiento

- El temporizador comienza con la primera rotación válida.
- Se actualiza cada segundo.
- Se detiene al ganar.
- Se detiene al perder.
- Se reinicia al reiniciar el nivel.
- No debe crear múltiples intervalos simultáneos.
- No debe continuar ejecutándose al cambiar de nivel.

Cuando llegue a cero:

- Bloquear las piezas.
- Mostrar el mensaje «Tiempo agotado».
- Mostrar los movimientos realizados.
- Permitir reintentar el nivel.

Utilizar un cálculo basado en tiempo transcurrido real para evitar acumulación de errores por retrasos de los intervalos.

Si el usuario cambia de pestaña, el tiempo debe seguir transcurriendo mientras la partida esté activa.

5. Sistema de puntuación

Implementar una puntuación sencilla y explicable.

```
Fórmula propuesta:
Puntuación = máximo entre 0 y (1000 + segundos restantes × 10 − movimientos × 20).
Requisitos:
- Calcular únicamente al completar un nivel.
- No otorgar puntos al perder.
- Mostrar el resultado en la pantalla de victoria.
- Evitar puntuaciones negativas.
- Mantener una puntuación individual por nivel.
- Evitar duplicar puntuaciones cuando se active repetidamente la condición de victoria.
Documentar claramente la fórmula.
## 6. Sistema de récords
Utilizar `localStorage` del navegador.
Guardar:
- Mejor puntuación por nivel.
- Menor cantidad de movimientos por nivel.
- Niveles desbloqueados.
Requisitos:
- Mantener los récords al recargar la página.
- Actualizarlos únicamente cuando se supere una marca.
- Validar los datos recuperados.
- Manejar correctamente datos ausentes o corruptos.
- Evitar que un error de almacenamiento impida jugar.
No implementar cuentas, servidores de almacenamiento ni bases de datos.
## 7. Selección de niveles
Incorporar una sección visual para seleccionar niveles.
Debe mostrar:
- Número y nombre del nivel.
- Estado bloqueado o desbloqueado.
- Mejor puntuación.
- Indicador del nivel seleccionado.
Reutilizar la identidad visual futurista.
Los niveles bloqueados no deben poder iniciarse mediante la interfaz ni mediante llamadas normales a las funciones de selección.
## 8. Pantallas de resultado
### Victoria
```

```
Mostrar:
- Nivel completado.
- Movimientos.
- Tiempo restante.
- Puntuación obtenida.
- Mejor récord.
- Botón para continuar al siguiente nivel, cuando exista.
- Botón para repetir el nivel.
### Derrota
Mostrar:
- Mensaje «Tiempo agotado».
- Nivel actual.
- Movimientos realizados.
- Botón «Reintentar».
Los resultados deben utilizar el sistema visual de modales
desarrollado en la Iteración 02.
Evitar modales duplicados.
## 9. Diseño visual y responsive
Conservar la estética futurista actual:
- Fondo azul oscuro.
- Circuitos cian.
- Fuente amarilla.
- Indicadores luminosos.
- Paneles compactos.
- Animaciones suaves.
Adaptar los tableros 5×5 y 6×6 para que sigan siendo utilizables
en dispositivos móviles.
No introducir desplazamiento horizontal.
Respetar `prefers-reduced-motion`.
Mantener controles accesibles mediante teclado.
No introducir bibliotecas visuales externas.
## 10. Arquitectura y calidad
Mantener Flask como servidor de la aplicación y JavaScript como
responsable de la lógica interactiva.
Si el crecimiento del código lo justifica, separar
responsabilidades en módulos JavaScript pequeños.
No realizar una refactorización general innecesaria.
Priorizar funciones claras para:
- Cargar niveles.
```

- Validar configuraciones.
- Rotar piezas.
- Propagar energía.
- Controlar el temporizador.
- Calcular puntuaciones.
- Gestionar récords.
- Cambiar de nivel.
- Reiniciar partidas.

Evitar dependencias adicionales.

11. Pruebas obligatorias

Conservar las siete pruebas existentes y adaptarlas únicamente cuando los cambios funcionales lo requieran.

Agregar pruebas para comprobar:

1. El nivel 01 conserva un tablero válido de 16 piezas.
2. El nivel 02 contiene 25 piezas.
3. El nivel 03 contiene 36 piezas.
4. Todos los niveles tienen al menos una solución.
5. Los niveles comienzan sin resolverse.
6. El temporizador no comienza antes del primer movimiento.
7. El temporizador disminuye correctamente.
8. El temporizador se detiene al ganar.
9. El temporizador se detiene al perder.
10. El tiempo agotado bloquea la interacción.
11. Reiniciar restaura el tiempo inicial.
12. No existen intervalos duplicados.
13. La puntuación coincide con la fórmula.
14. La puntuación nunca es negativa.
15. Los récords se actualizan correctamente.
16. Los récords persisten al recargar.
17. Los datos corruptos de almacenamiento se manejan sin errores críticos.
18. Los niveles bloqueados no pueden iniciarse.
19. Completar un nivel desbloquea el siguiente.
20. La interfaz funciona en escritorio y móvil.

No declares aprobadas pruebas que no hayan sido ejecutadas.

Documenta resultados reales y limitaciones de verificación.

12. Documentación académica

La actividad de KODIGO evalúa:

- Funcionamiento: 2.5 puntos.
- Uso estratégico de IA: 2.5 puntos.
- Iteración y mejora: 3 puntos.

- Comprensión y validación: 2 puntos.

Actualizar `docs/iteraciones.md` con:

Iteración 03 — Ampliación de jugabilidad

Incluir:

1. Objetivo.
2. Justificación de las nuevas funcionalidades.
3. Prompt utilizado.
4. Archivos modificados.
5. Cambios funcionales.
6. Pruebas ejecutadas.
7. Resultados obtenidos.
8. Errores reales encontrados.
9. Correcciones aplicadas.
10. Limitaciones pendientes.

En `docs/datos/info.md`, incorporar íntegramente este Prompt 03, conservando los anteriores y su orden cronológico.

Actualizar el README con las instrucciones de juego y las nuevas funcionalidades.

No inventar resultados ni reflexiones personales.

13. Evidencias

Crear o utilizar la carpeta:

`docs/assets/contenido/evidencias/iteracion-03/`

Evidencias sugeridas:

- Captura 09: selector de niveles.
- Captura 10: nivel 02 con tablero 5×5.
- Captura 11: nivel 03 con tablero 6×6.
- Captura 12: temporizador en funcionamiento.
- Captura 13: pantalla de victoria con puntuación.
- Captura 14: pantalla de derrota.
- Captura 15: resultados de pruebas automatizadas.

Generar capturas únicamente si existen herramientas disponibles para hacerlo realmente.

Si no puedes capturarlas, indicar cuáles debo obtener manualmente.

14. Metodología de ejecución

Trabaja en cuatro fases:

Fase A — Análisis: inspeccionar el código y ejecutar pruebas de referencia.

Fase B — Implementación: incorporar niveles, temporizador, puntuación y récords.

Fase C — Validación: ejecutar pruebas y corregir errores reales.

Fase D — Documentación: actualizar README, info.md e iteraciones.md y registrar evidencias.

Al finalizar, entregar un resumen técnico con:

- Funcionalidades implementadas.
- Archivos modificados.
- Pruebas ejecutadas y resultados.
- Errores corregidos.
- Evidencias generadas.
- Limitaciones conocidas.
- Instrucciones para ejecutar y probar la aplicación.

15. Restricciones finales

- No reconstruir el proyecto.
- No alterar innecesariamente el diseño de la Iteración 02.
- No utilizar frameworks adicionales.
- No incorporar bases de datos.
- No introducir funcionalidades ajenas al alcance.
- No realizar commits ni pushes automáticamente.
- No avanzar hacia una cuarta iteración sin autorización.
- Priorizar claridad, estabilidad y comprensión del código.

Comienza analizando el proyecto existente y desarrolla exclusivamente la Iteración 03.

